



Strathmore University

@iLabAfrica Centre

Session 7: Computer Vision with CNNs Transfer Learning & Fine-Tuning

Javier Serrano

Applied Machine Learning

Master in Data Science & Analytics

Session objectives

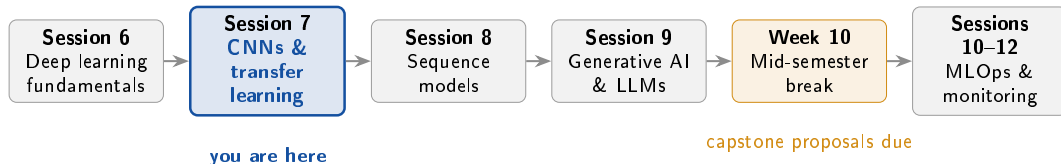
What you will be able to do after today



1. **Explain** why convolution is the right inductive bias for images.
2. **Compute** feature-map shapes and parameter counts for convolutional stacks.
3. **Describe** residual learning (ResNet) and compound scaling (EfficientNet).
4. **Apply** transfer learning: frozen-backbone feature extraction, then staged fine-tuning.
5. **Assess** deployment and governance constraints for vision models in Kenyan fintech.

Where this session sits

From general networks to specialised architectures



Session 6 gave us dense networks. Images have **structure** that a dense network throws away, today we build the architecture that keeps it, and learn to **reuse** networks trained by others.

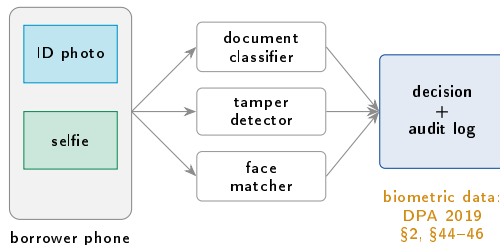
The applied thread

KYC document verification for a Nairobi lender



A digital credit provider onboards a borrower in under three minutes. The vision system must:

- classify the uploaded **ID document type**;
- detect **tampering** (splicing, re-photography, screen capture);
- match the **selfie** to the ID portrait.



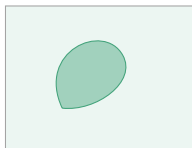
Wrong accept $\Rightarrow$ fraudulent loan. Wrong reject $\Rightarrow$ a legitimate thin-file borrower is locked out.

Four vision tasks, one backbone

What changes is the head, not the feature extractor

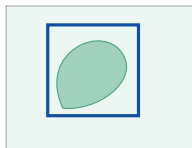


classification



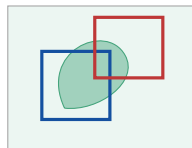
one label per image

localisation



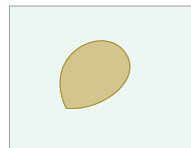
label + box

detection



many boxes

segmentation



label per pixel

Today we treat **classification**. The convolutional *backbone* is shared across all four, which is exactly why a backbone trained on one task transfers to another.

Roadmap

Session 7 – Computer Vision with CNNs



Why Convolutions?

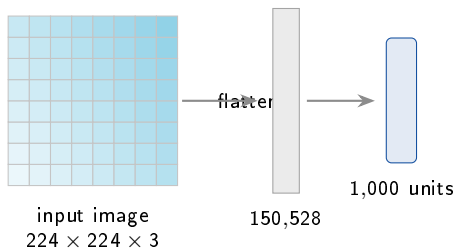
Modern Architectures: ResNet and EfficientNet

Transfer Learning

Evaluation, Explainability and Deployment

Problem 1: the dense layer explodes

A 224×224 RGB image has 150,528 input features

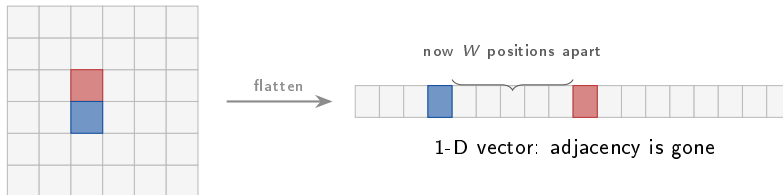


one weight matrix
 $150,528 \times 1,000 \approx$
 1.5×10^8 parameters

That is a single hidden layer, more parameters than ResNet-50 has in total, before any depth is added. Memorisation is guaranteed; generalisation is not.

Problem 2: flattening destroys geometry

Neighbouring pixels become unrelated coordinates



2-D image: the two pixels touch

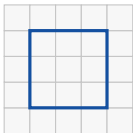
A dense layer would have to *re-learn* that these two coordinates are neighbours, from data, for every pair, at every location. We should build that knowledge into the architecture instead.



The three inductive biases of convolution

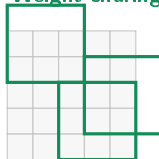
Assumptions about images, baked into the architecture

1. Locality



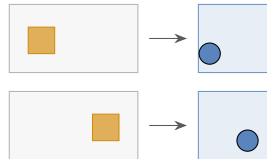
a unit sees a small patch,
not the whole image

2. Weight sharing



the *same* kernel is
applied everywhere

3. Translation equivariance



shift the input $\Rightarrow$ the
response shifts too

An ID number in the top-left and the same number in the centre are the **same evidence**.
Convolution encodes that; a dense layer does not.

The convolution operation

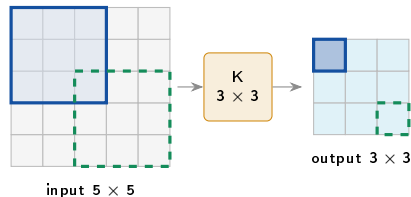
A learned kernel slid across the input



For input X and kernel K of size $f \times f$, the (cross-correlation) response at (i, j) is

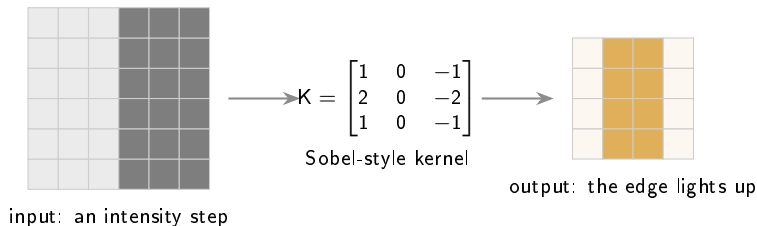
$$S_{i,j} = \sum_{u=0}^{f-1} \sum_{v=0}^{f-1} X_{i+u, j+v} K_{u,v} + b.$$

- The kernel entries are **learned**, not designed.
- One kernel $\Rightarrow$ one output *feature map*.
- f^2 weights, reused at every position.



What a single kernel detects

The classical vertical-edge filter, before learning replaced it



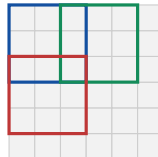
In a CNN we never write these numbers down. Back-propagation discovers the kernels that minimise the loss, and the first-layer kernels it finds look remarkably like edge and colour-blob detectors.

The two knobs that control output geometry



A 5x5 grid of squares. The central 3x3 subgrid is highlighted in light blue, while the outer squares are light orange. This represents a 3x3 convolution kernel applied to a 5x5 input.

"same" pads a border of zeros so the output keeps the input size;
"valid" does not pad and shrinks



the kernel jumps s pixels;
 $s = 2$ halves the resolution
 and quarters the compute

Output size: the formula you will use constantly

Get this wrong and the model silently mis-shapes



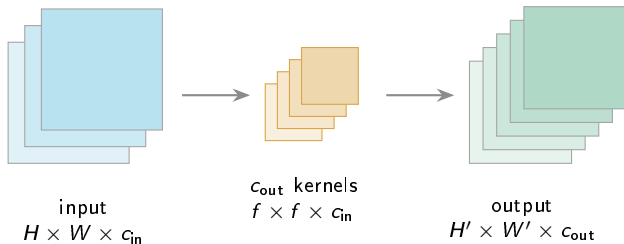
$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - f}{s} \right\rfloor + 1$$

Configuration	n_{in}	f	p	s	n_{out}
<code>Conv2D(..., 3, padding="same")</code>	224	3	1	1	224
<code>Conv2D(..., 3, padding="valid")</code>	224	3	0	1	222
<code>Conv2D(..., 7, strides=2, padding="same")</code>	224	7	3	2	112
<code>MaxPooling2D(2)</code>	112	2	0	2	56

Note the last two rows: they are the first two operations of ResNet-50. Resolution falls $224 \rightarrow 56$ before the residual stages even begin.

Channels: the third dimension

A convolutional layer maps a tensor to a tensor



each kernel spans
all input channels
and produces **one**
output channel

Parameter counting

A skill you will use every time you read a model summary



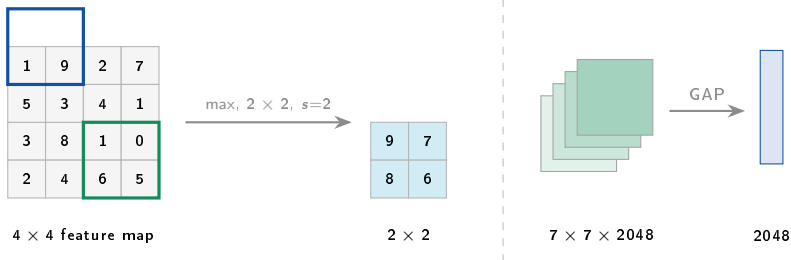
$$\# \text{params} = \underbrace{f^2 c_{\text{in}} c_{\text{out}}}_{\text{kernels}} + \underbrace{c_{\text{out}}}_{\text{biases}}$$

Layer	Output shape	Parameters	Note
Input	$224 \times 224 \times 3$	0	RGB document photo
Conv2D(32, 3)	$224 \times 224 \times 32$	896	vs. 1.5×10^8 dense
MaxPooling2D(2)	$112 \times 112 \times 32$	0	spatial downsample
Conv2D(64, 3)	$112 \times 112 \times 64$	18,496	channels grow as H, W shrink
GlobalAvgPool2D	64	0	replaces Flatten
Dense(5, softmax)	5	325	the classifier head

Parameter cost is **independent of image size**; activation memory is not.

Pooling: shrink space, keep evidence

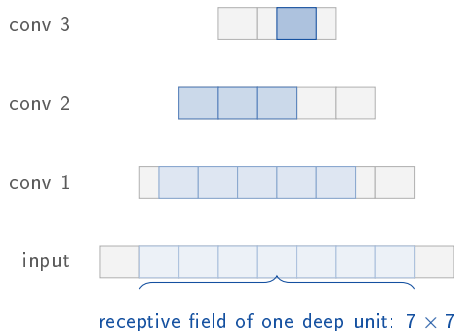
And global average pooling, which kills the parameter tail



Global average pooling averages each channel over all positions. It removes the flatten-then-dense bottleneck, makes the network robust to input size, and is what every modern backbone uses.

The receptive field grows with depth

Why deep stacks of small kernels beat shallow stacks of large ones



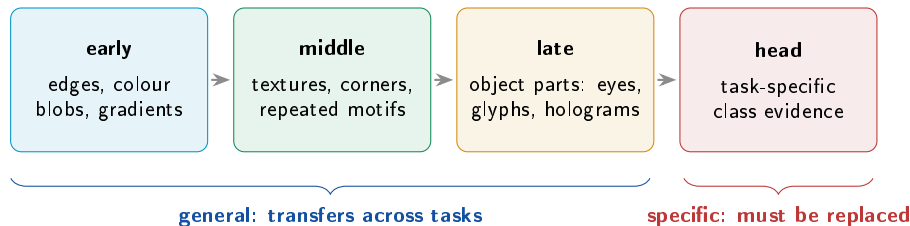
Three 3×3 layers see a 7×7 window using $3f^2 = 27$ weights per channel pair.

One 7×7 layer sees the same window using 49, and inserts **no** non-linearities in between.



The feature hierarchy

What the layers actually end up representing



This single picture is the justification for everything in the second half of the session.

A baseline CNN in tf.keras

Conv → BN → ReLU → pool, three times



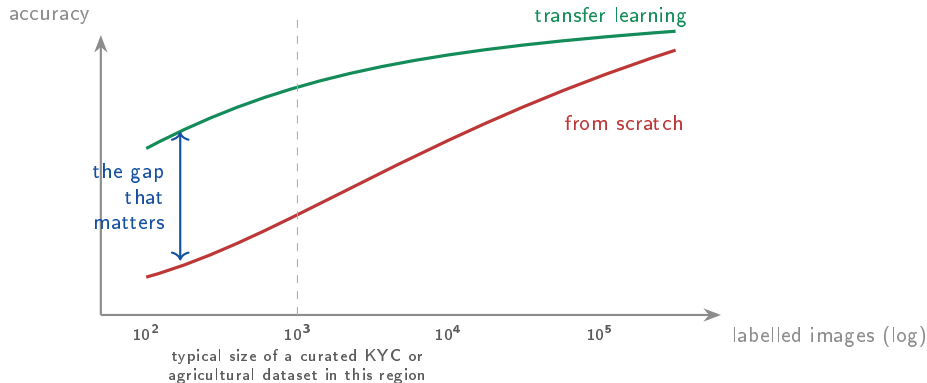
```
from tensorflow.keras import layers, models

def baseline_cnn(n_classes=5, input_shape=(224, 224, 3)):
    inputs = layers.Input(shape=input_shape)
    x = layers.Rescaling(1.0 / 255)(inputs)
    for filters in (32, 64, 128):
        x = layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
        x = layers.BatchNormalization()(x)    # bias is redundant before BN
        x = layers.Activation("relu")(x)
        x = layers.MaxPooling2D(2)(x)         # halve H and W
        x = layers.GlobalAveragePooling2D()(x) # (H, W, C) -> (C,)
        x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(n_classes, activation="softmax")(x)
    return models.Model(inputs, outputs, name="baseline_cnn")
```

About 130k parameters, three orders of magnitude below the single dense layer we started with.

Why the baseline will still disappoint

Small labelled datasets are the norm, not the exception



You will rarely have ImageNet-scale labels. You will almost always have access to a network that was trained on them.



Roadmap

Session 7 – Computer Vision with CNNs

Why Convolutions?

Modern Architectures: ResNet and EfficientNet

Transfer Learning

Ten years of architecture search

Every entry is a pre-trained backbone you can download today

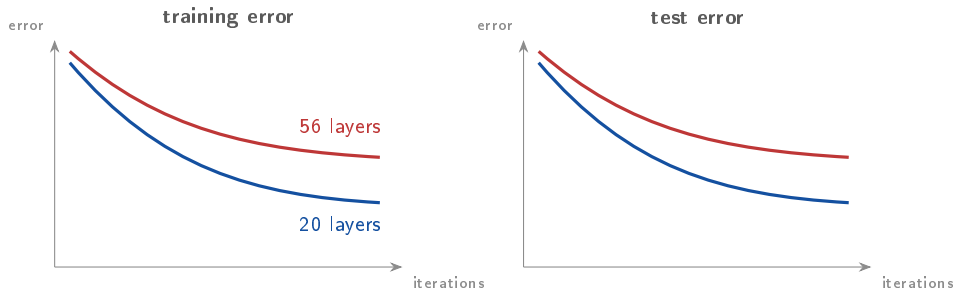


Year	Model	Params	Top-1	Idea introduced
1998	LeNet-5	60 k	–	convolution + pooling
2012	AlexNet	60 M	63 %	ReLU, dropout, GPU training
2014	VGG-16	138 M	74 %	uniform 3×3 stacks
2014	GoogLeNet	6.8 M	74 %	multi-scale inception blocks
2015	ResNet-50	25.6 M	76 %	residual connections
2019	EfficientNet-B0	5.3 M	77 %	compound scaling
2021	EfficientNetV2-S	21.5 M	84 %	fused MBConv, progressive training

Notice the trend: accuracy rises while parameter count *falls*. That matters when the model has to run on a mid-range Android handset in Nairobi.

The degradation problem

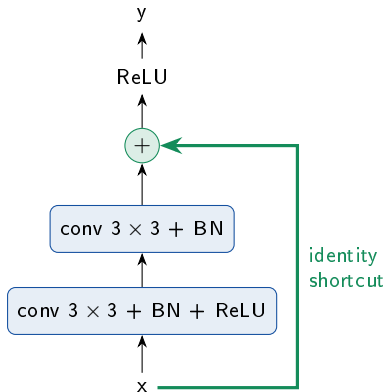
Deeper networks got *worse*, and not because of overfitting



The 56-layer plain network is worse on **training** error too, so this is not overfitting, it is an **optimisation** failure. A deeper model can always represent a shallower one by making the extra layers the identity, yet gradient descent could not find that solution.

The residual block

Make the identity the default, learn only the deviation



The block computes

$$y = \mathcal{F}(x; \theta) + x.$$

- If the optimal mapping *is* the identity, the network only needs $\mathcal{F} \rightarrow 0$, easy, since weight decay already pushes there.
- The shortcut carries **no parameters**.
- When shapes differ, a 1×1 projection conv is inserted on the shortcut path.

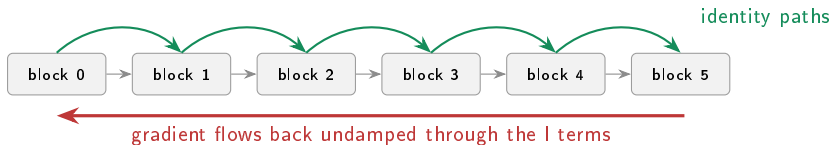
Why the shortcut fixes the gradient

A highway from the loss to every early layer



Differentiating $y = \mathcal{F}(x) + x$,

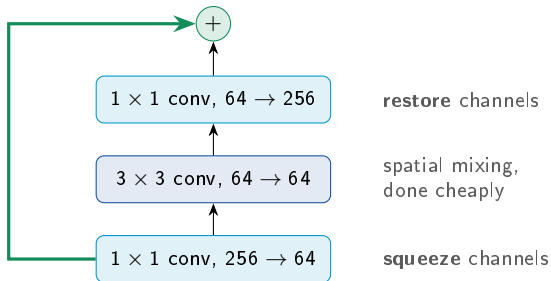
$$\frac{\partial y}{\partial x} = \frac{\partial \mathcal{F}}{\partial x} + 1 \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial x_\ell} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=\ell}^{L-1} \left(1 + \frac{\partial \mathcal{F}_k}{\partial x_k} \right).$$



The product no longer collapses to zero: even if every $\partial \mathcal{F}_k / \partial x_k$ is tiny, the identity keeps the signal alive. This is what made 50-, 101- and 152-layer networks trainable.

The bottleneck block

How ResNet-50 keeps depth affordable

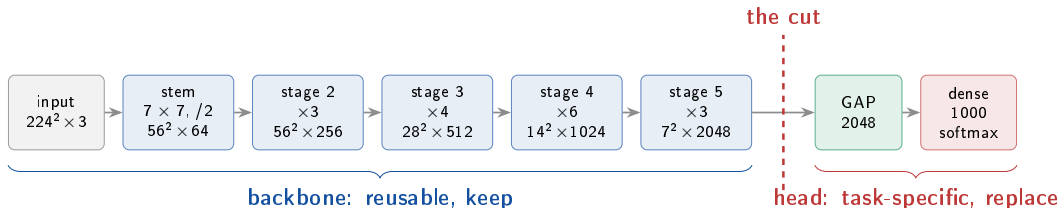


A naive 3×3 conv at $256 \rightarrow 256$ costs $\approx 590k$ weights.

The bottleneck triple costs $\approx 70k$, an $8\times$ saving, per block, $\times 16$ blocks.

ResNet-50 end to end

And the single most important line in this session



`include_top=False` in Keras gives you everything to the left of the red line.

EfficientNet: where should extra compute go?

Depth, width, or resolution, the answer is all three



baseline



depth d



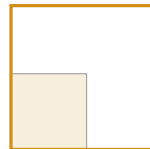
more layers: richer features, harder to train

width w



more channels: fine detail, saturates fast

resolution r



bigger input: small objects survive, cost is quadratic

Scaling any single dimension gives diminishing returns and plateaus at around 80 % top-1. Tan & Le (2019) showed the three must be scaled **jointly and in fixed proportion**.

Compound scaling

One knob, ϕ , moves all three dimensions



$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi \quad \text{subject to} \quad \alpha \beta^2 \gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1.$$

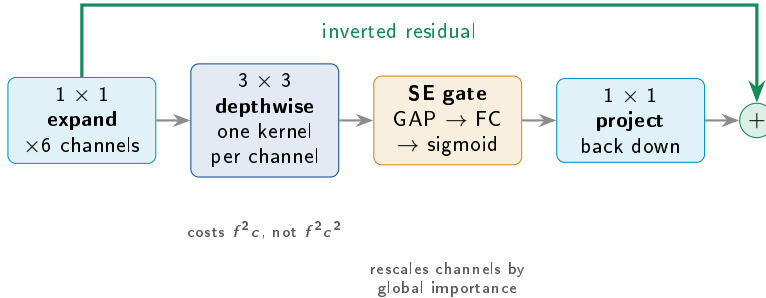
- Width and resolution enter **quadratically** in FLOPs, depth linearly, hence the exponents in the constraint.
- The constraint makes each increment of ϕ roughly **double** the compute budget.
- (α, β, γ) are found *once* by a small grid search on the B0 base network; ϕ then generates the whole B0–B7 family.



5.3M → 66M params; 77 % → 84 %
top-1

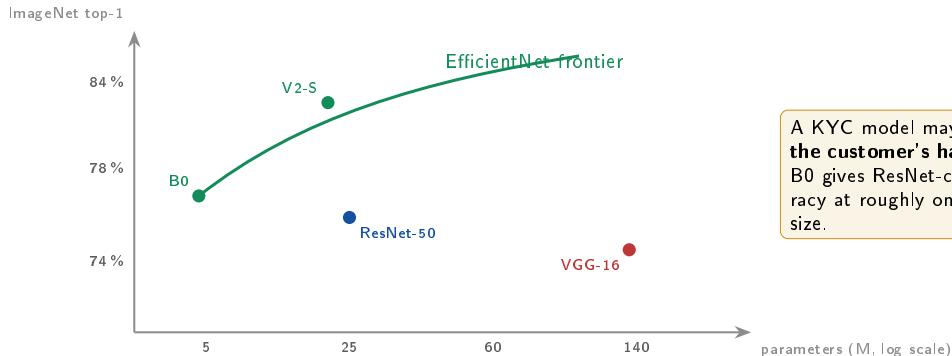
Inside EfficientNet: MBConv with squeeze-and-excitation

Cheap spatial mixing plus a learned channel gate



The accuracy–size frontier

Why B0 is often the right answer in a mobile-first market



A KYC model may run **on the customer's handset**. B0 gives ResNet-class accuracy at roughly one fifth the size.

Choosing a backbone

A decision table you can defend in a design review

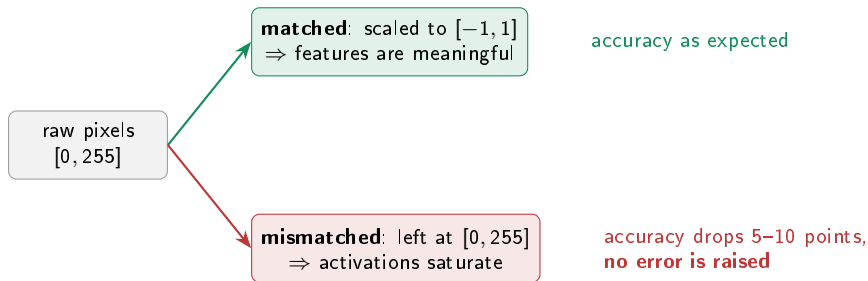


Criterion	ResNet50V2	EfficientNetV2-B0
Params / size	25.6M, ~98 MB	7.2M, ~26 MB
Accuracy per FLOP	good	better
Fine-tuning behaviour	robust, forgiving	excellent, slightly more LR-sensitive
Preprocessing	resnet_v2 module, range $[-1, 1]$	built into the model , pass raw $[0, 255]$
Edge deployment	heavy without compression	designed for it

Rule of thumb: prototype with ResNet50V2 because it is predictable; ship with EfficientNetV2 when size or latency is a constraint. Better still, run both and compare on your own data.

Pitfall: the preprocessing mismatch

The most common silent bug in transfer learning



Always take the preprocessing function from the *same* module as the backbone:
`keras.applications.resnet_v2.preprocess_input`, and so on. If you switch backbones,
switch the preprocessing with it.

Roadmap

Session 7 – Computer Vision with CNNs



Why Convolutions?

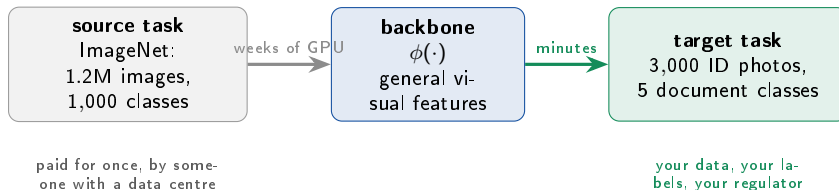
Modern Architectures: ResNet and EfficientNet

Transfer Learning

Evaluation, Explainability and Deployment

The core idea

Someone else already paid for the general features

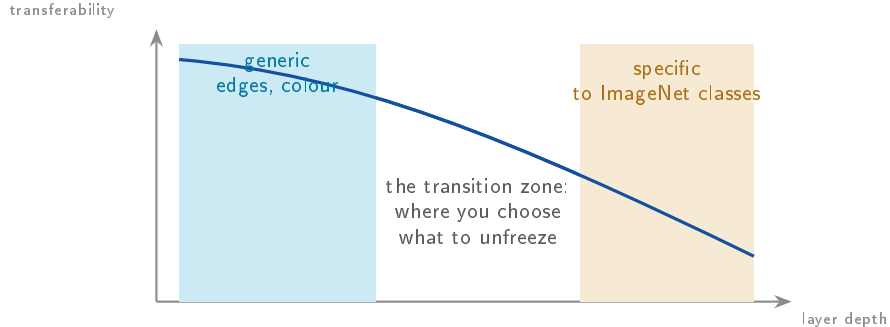


Formally: we transfer ϕ_{source} and learn only a new head g , so that $\hat{y} = g(\phi(x))$, optionally letting ϕ drift a little afterwards.



Why it works: transferability decays with depth

Yosinski et al. (2014)



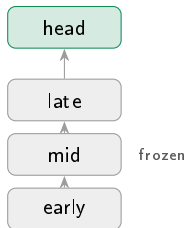
The closer a layer is to the input, the more its features are properties of *natural images in general* rather than of the source labels. Early layers transfer almost for free; late layers must be adapted or replaced.

Two regimes

Feature extraction and fine-tuning

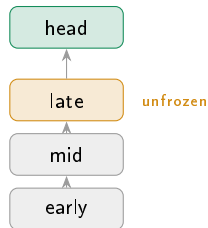


feature extraction



small data, similar
domain; minutes on CPU

fine-tuning

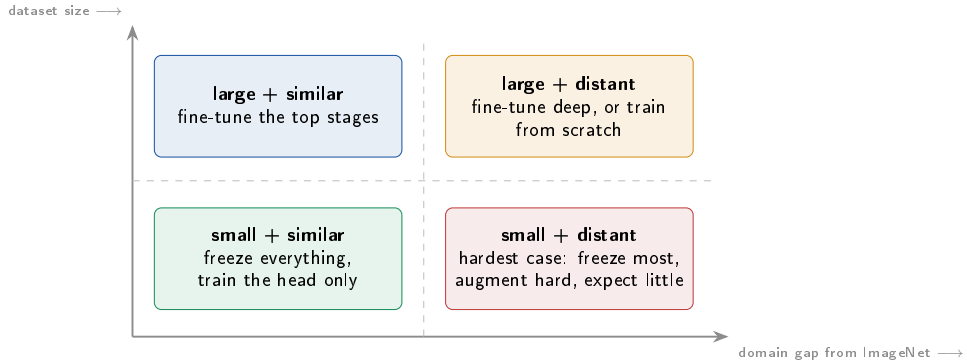


more data or a domain
gap; needs a GPU

In practice you do **both**, in that order. Feature extraction first is not optional, the next slide explains why.

Which regime? Read it off two axes

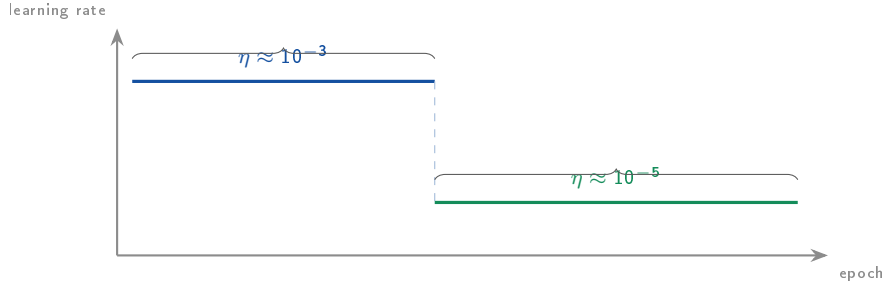
Dataset size against domain gap



Satellite and X-ray imagery sit on the right; photographs of documents, faces and crops sit on the left. Most African fintech vision problems are **bottom-left**.

The two-phase recipe

Order matters more than any hyper-parameter here

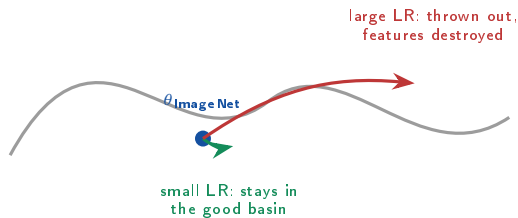


Phase A: backbone frozen
train the new head
to convergence

Phase B: top stages unfrozen
adapt late features gently

Why not unfreeze on step one?

Catastrophic forgetting, drawn



A randomly initialised head emits **large, meaningless gradients**. If the backbone is unfrozen at step one, those gradients propagate down and overwrite the pre-trained weights before the head has learned anything.

The 10–100 $\times$ learning-rate reduction in Phase B has the same purpose: we want to *nudge* θ away from $\theta_{ImageNet}$, not re-derive it.

Discriminative learning rates

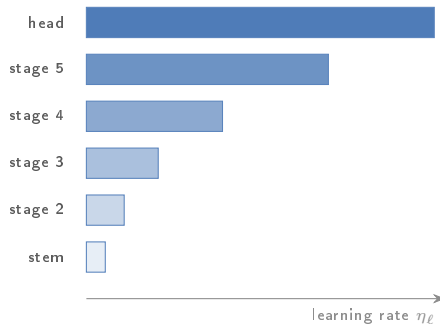
Deeper layers should move less than shallower ones



A refinement of the two-phase recipe:
instead of one rate for the whole unfrozen
region, set

$$\eta_{\ell} = \eta_{\text{top}} \cdot \xi^{L-\ell}, \quad \xi \in [0.3, 0.9].$$

- Generic early layers barely move.
- Late layers adapt to the target domain.
- In Keras: per-layer LR multipliers, or simply progressive unfreezing stage by stage.



Phase A in tf.keras

Frozen backbone, new head



```
from tensorflow import keras
from tensorflow.keras import layers

backbone = keras.applications.ResNet50V2(
    include_top=False, weights="imagenet", input_shape=(224, 224, 3))
backbone.trainable = False                # freeze every weight
inputs = layers.Input(shape=(224, 224, 3))
x = augment(inputs)                       # augmentation layers
x = keras.applications.resnet_v2.preprocess_input(x)
x = backbone(x, training=False)           # keep BN in inference mode
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(N_CLASSES, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

Phase B in tf.keras

Unfreeze the top stage, drop the learning rate



```

backbone.trainable = True                                # unfreeze the whole backbone...
    for layer in backbone.layers:                        # ...then re-freeze all but conv5
        if not layer.name.startswith("conv5"):
            layer.trainable = False
        if isinstance(layer, layers.BatchNormalization):
            layer.trainable = False                      # never update BN statistics

model.compile(optimizer=keras.optimizers.Adam(1e-5),    # 100x smaller
loss="sparse_categorical_crossentropy", metrics=["accuracy"])

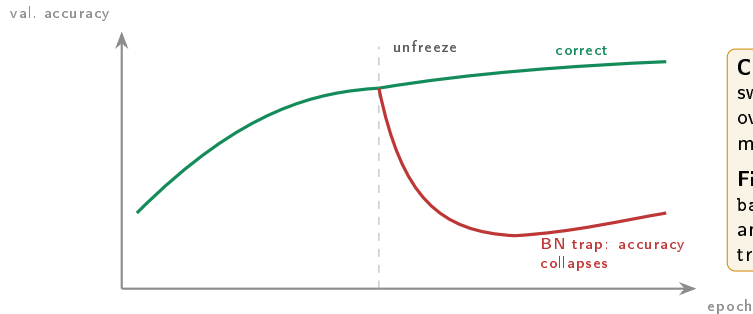
callbacks = [
keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True,
monitor="val_loss"),
keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]

```

Re-compiling is mandatory: Keras caches the trainable-weight list at compile time.

Pitfall: the BatchNorm trap

The single most common transfer-learning bug

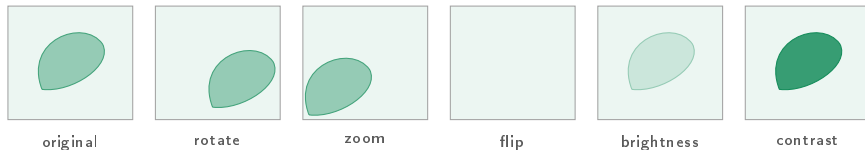


Cause: BatchNorm layers switch to batch statistics and overwrite the ImageNet running means.

Fix: call the backbone as `backbone(x, training=False)` and keep BN layers non-trainable.

Data augmentation

Multiplying a small dataset with label-preserving transforms



Augmentation minimises $\mathbb{E}_{(x,y)} \mathbb{E}_{T \sim \mathcal{T}} [\mathcal{L}(f_{\theta}(T(x)), y)]$: a regulariser that encodes the invariances the label genuinely has. In Keras these are *layers* inside the model, active during fit, inert during predict.

Augmentation is a domain decision, not a default

The same transform can be correct or catastrophic



Crop disease photograph

a mirrored leaf is
still a diseased leaf

horizontal flip: correct

ID document photograph

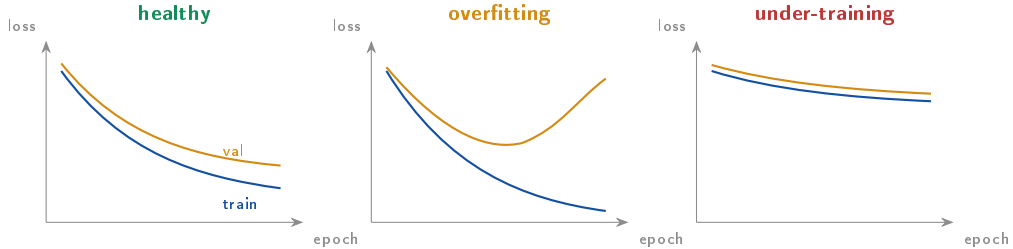
mirrored text is a *tamper signal*,
and the label is “genuine”

horizontal flip: teaches the model to ignore fraud

The augmentation policy encodes what you believe is irrelevant to the label. Get that belief wrong and you have injected label noise on purpose. Always write the policy down and justify each transform.

Reading the training curves

Three signatures you will meet repeatedly



Overfitting → stronger augmentation, more dropout, freeze more, stop earlier.

Under-training → unfreeze more, raise the learning rate, train longer. A **sudden collapse at the unfreeze point** is almost always the BatchNorm trap.



Roadmap

Session 7 – Computer Vision with CNNs

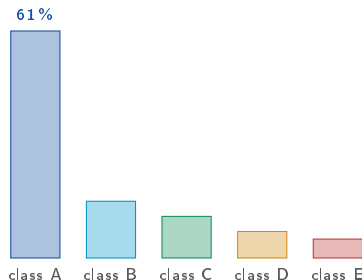
Why Convolutions?

Transfer Learning

Evaluation, Explainability and Deployment

Accuracy is the wrong metric again

Imbalance in vision looks exactly like imbalance in credit scoring



A model that always predicts class A scores 61 % accuracy and is worthless.

Report **macro-F1**, the per-class confusion matrix, and per-class recall, exactly the discipline from Session 3.

$\text{macro-F1} = \frac{1}{K} \sum_{k=1}^K \frac{2 P_k R_k}{P_k + R_k}$ weights every class equally, so a rare-but-costly class cannot be ignored.

Handling imbalance in vision

Class weights, not resampling



```
import numpy as np
from sklearn.utils.class_weight import compute_class_weight

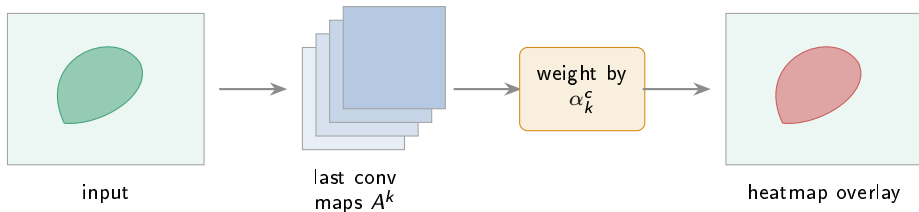
weights = compute_class_weight("balanced",
                               classes=np.unique(y_train), y=y_train)
class_weight = dict(enumerate(weights))

model.fit(train_ds, validation_data=val_ds,
          epochs=EPOCHS, class_weight=class_weight, callbacks=callbacks)
```

Why not SMOTE? Synthetic interpolation between two images in pixel space produces a ghost, not a plausible image. In vision, *augmentation of the minority class* plays the role that SMOTE plays for tabular data, which is one more reason the augmentation policy deserves care.

Grad-CAM: is the model looking at the right thing?

Class-discriminative localisation, for free



$$\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial y^c}{\partial A_{i,j}^k}, \text{ then } L^c = \text{ReLU}(\sum_k \alpha_k^c A^k), \text{ upsampled to input resolution.}$$

Grad-CAM in twelve lines

Two forward passes and one tape



```
import tensorflow as tf

def grad_cam(model, image, class_index, last_conv="conv5_block3_out"):
    grad_model = tf.keras.Model(
        model.inputs,
        [model.get_layer(last_conv).output, model.output])
    with tf.GradientTape() as tape:
        fmaps, preds = grad_model(image[None, ...])
    score = preds[:, class_index]
    alpha = tf.reduce_mean(tape.gradient(score, fmaps), axis=(1, 2))
    cam = tf.nn.relu(tf.einsum("bk,bijk->bij", alpha, fmaps))
    return cam / (tf.reduce_max(cam) + 1e-8)
```

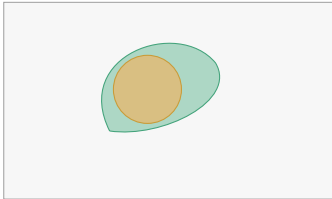
Use it as an **audit**, not a decoration: sample the model's confident errors and check where it was looking.

Shortcut learning

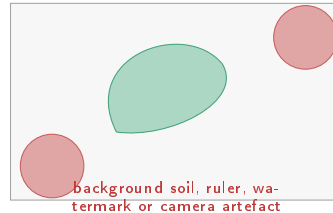
The failure Grad-CAM is designed to catch



what we hoped



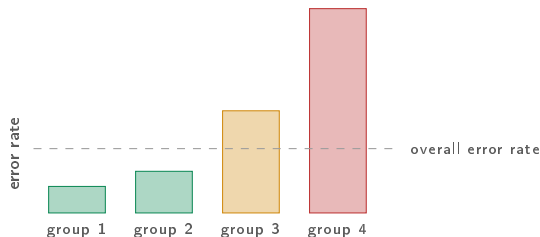
what it learned



A model can reach high validation accuracy by keying on the **data-collection process** rather than the phenomenon: the clinic that photographed the sick plants used a different phone; the fraudulent IDs were all scanned on the same device. Such a model collapses on deployment day.

Fairness in vision systems

Aggregate accuracy hides who is failing



Buolamwini & Gebru (2018) found commercial face classifiers with near-perfect accuracy overall and error rates above 30 % for darker-skinned women.

The aggregate number was never wrong, it was answering the wrong question.

Practice: stratify every evaluation by skin tone, age band, gender, device tier and lighting condition. Publish the stratified table, not the headline number.

Kenyan regulatory context

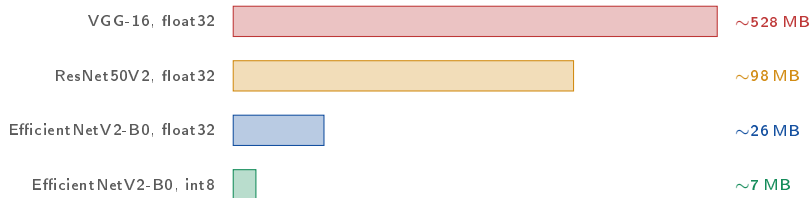
Biometric vision systems are squarely in scope



- **DPA 2019 §2:** facial images and other biometric identifiers are **sensitive personal data**, explicit consent and a lawful basis are required.
- **DPA 2019 §25 & §30:** purpose limitation and data minimisation. Selfies collected for identity verification may not be silently reused to train a general face model.
- **DPA 2019 §35:** a data subject has the right not to be subject to a decision based solely on automated processing, so a **human review path** for rejected applicants is not optional.
- **CBK Digital Credit Providers Regulations 2022:** model governance, auditability and complaint handling for licensed lenders.
- **Practical consequence:** store the model version, the input hash, the Grad-CAM artefact and the reviewer decision for every rejection.

From notebook to handset

Size and latency are product requirements, not afterthoughts



Post-training **int8 quantisation** in TFLite cuts size roughly fourfold and speeds up inference on ARM, usually for under one point of accuracy. On a network where a borrower pays for every megabyte, that is the difference between an app people install and one they do not.

Where this leads

The vision model is one node in a larger system

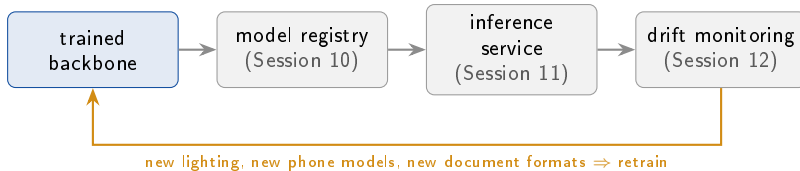


Image distributions drift for mundane reasons: a handset generation changes the camera pipeline, a government reissues an ID design, the rainy season changes field lighting. Monitoring closes the loop.

Key takeaways

Session 7 in six lines



1. Convolution is not a trick, it is an **inductive bias**: locality, weight sharing, translation equivariance.
2. **Residual connections** made depth trainable; **compound scaling** made it efficient.
3. Early layers are generic and transfer; late layers are task-specific and must be adapted.
4. **Freeze, train the head, then unfreeze with a much smaller learning rate.** Order is not negotiable.
5. The two silent killers are the **preprocessing mismatch** and the **BatchNorm trap**.
6. Evaluate per class and per demographic group, explain with Grad-CAM, and design for the handset and the regulator from the start.

Reading

Primary sources for Chapter 7



- Géron, *Hands-On Machine Learning*, 3rd ed., Ch. 14 (2nd ed. Ch. 14), CNNs and Keras applications.
- Bishop, *Deep Learning: Foundations and Concepts*, Ch. 10, convolutional networks and equivariance.
- He et al. (2016), *Deep Residual Learning for Image Recognition*, CVPR.
- Tan & Le (2019), *EfficientNet*, ICML; Tan & Le (2021), *EfficientNetV2*, ICML.
- Yosinski et al. (2014), *How Transferable are Features in Deep Neural Networks?*, NeurIPS.
- Selvaraju et al. (2017), *Grad-CAM*, ICCV.
- Buolamwini & Gebru (2018), *Gender Shades*, FAccT.
- Republic of Kenya, *Data Protection Act* (2019); CBK, *Digital Credit Providers Regulations* (2022).

Session 8: Sequence Models

Recurrent networks, LSTMs and temporal fusion for transaction streams



Dr GOLIARDO said

jgarcia@strathmore.edu